

SECTION 22 — PARALLEL TESTING AND SCALABILITY (Refined)

Topics Covered

- 22.1-22.18 (18 headers)

First pass — 2 parallel batch subagents

22.1 Why Parallelize — Refined

Theory: Parallel execution cuts suite time from hours to minutes by running tests across workers (processes), files, and machines simultaneously. Faster CI feedback → quicker releases → higher throughput with zero coverage loss. Playwright runs files in parallel by default via workers; Selenium Grid distributes WebDriver sessions remotely. Speedup is near-linear until infrastructure bottlenecks (CPU, DB, app capacity); Amdahl's law bounds it by the serial fraction. Enterprise Relevance: A 2h serial regression blocking every merge is a delivery freeze. Deloitte pattern: 4 workers + 4 shards turns 2h into ~10min; PR feedback <10min is the adoption threshold — slower and devs bypass gates.

```
npx playwright test --workers=4 --fully-parallel
# Selenium: TestNG parallel="methods" thread-count="4" + Grid 4 nodes
```

High-Stakes Scenario: Release train waits on 3h nightly; hotfix cannot ship. Triage: profile serial fraction (setup, slowest file), parallelize files first (free), then methods/shards, then split smoke (PR) from full (nightly). Measure $S=T1/TN$ each step; stop when efficiency $E=S/N$ drops below 0.5. Anti-Patterns: Parallelizing order-dependent tests (failures multiply); adding workers before isolation (flake storm); quoting "4 workers = 4x" without measuring (Amdahl). Qs: What bounds parallel speedup? When does parallelization hurt instead of help?

22.2 Test Independence — Refined

Theory: Independent tests run in any order, on any worker, without depending on another test's output. Each test sets up its own preconditions via fixtures/APIs/fresh browser contexts and cleans up afterwards. No shared globals, no ordered assumptions, fresh browser per test. Independence enables parallelism, retries, and sharding safely — the prerequisite for everything else in this section. Enterprise Relevance: Order-dependent suites cannot be parallelized, retried, or sharded; one failure cascades. Deloitte gate: every new test must pass alone, shuffled, and repeated before merge.

```
test.describe.configure({ mode: 'parallel' });
test('isolated', async ({ page, request }) => {
  const user = await request.post('/users', { data: { email: `u-${Date.now()}@t.com` } })
    .then(r => r.json());
  await page.goto(`/users/${user.id}`);
  await expect(page.getByTestId('email')).toHaveText(user.email);
});
```

High-Stakes Scenario: Suite passes serially, 30% fails parallel — Test B reads row Test A created. Triage: run each failing test alone (passes → dependence confirmed), `--workers=1` vs 4 diff, forbid

cross-test reads, seed per test via API, randomized order in CI to catch new dependencies. Anti-Patterns: Login once in `beforeAll` shared across tests; hardcoded "test user 42"; `dependsOnMethods` chains; Suite → Test data handoff via files. Qs: How do you prove a test is independent? What breaks first when independence is violated?

22.3 Shared State Problems — Refined

Theory: Shared state is anything outside one test: static variables, singletons, files, cookies, `localStorage`, global accounts, external services. In parallel workers these cause flakiness, order-dependence, and cross-talk because OS processes cannot safely share memory. Signature symptom: passes serially, fails in parallel. Playwright isolates `BrowserContext` per worker (frontend), but backend state (DB rows, accounts) remains risky. Fix with fixtures and locks (`lock: 'user-settings'` for unavoidable shared resources). Enterprise Relevance: Shared state is the #1 parallelization blocker. Deloitte triage rule: serial-green + parallel-red = shared state until proven otherwise. Every shared resource needs an owner, a lock, or elimination.

```
// Serialize unavoidable shared access; isolate everything else
test('update settings', { lock: 'user-settings' }, async ({ page }) => { /* ... */
});
```

High-Stakes Scenario: 4 workers, all tests use `admin@test.com` — login evictions, profile overwrites, 25% flake. Triage: `workerIndex`-scoped users (`admin-w${workerIndex}`), per-worker `storageState`, no shared singletons; verify with `--workers=1` vs 4 delta → zero after fix. Anti-Patterns: Static `WebDriver` /token/config mutated per test; shared download path; global counter files; singleton pages reused across workers. Qs: Serial-pass/parallel-fail differential diagnosis? What state does `BrowserContext` isolate vs not?

22.4 Data Collisions — Refined

Theory: Data collisions occur when two parallel tests create, read, or delete the same record — same `orderId`, email, or CSV path. Results: overwrites, unique-constraint errors, false failures. Prevention: generate unique data per test from `testInfo.testId`, use `testInfo.outputPath()` for files, prefer API seeding over UI reuse. For shared datasets, scope creation per worker. Enterprise Relevance: The most common parallel flake signature: duplicate key value violates unique constraint "users_email_key". Deloitte standard: no hardcoded emails/IDs anywhere; uniqueness derived from run + worker + test IDs.

```
const orderId = `order-${testInfo.testId}`;
await fs.promises.writeFile(testInfo.outputPath('export.csv'), data); // per-test
path
const email = `qa-${Date.now()-w${testInfo.workerIndex}@test.com`;
```

High-Stakes Scenario: 8 parallel workers, all create `test@example.com` → 7 fail duplicate-key, suite 12% pass. Triage: unique emails (UUID/testId), per-worker DB schemas or `WHERE run_id`, file output per test (`outputPath`), shared-dataset tests serialized with lock. Anti-Patterns: Hardcoded emails/IDs; shared CSV path; `TRUNCATE` in parallel (wipes others' data); asserting on "latest row". Qs: How do you make every test's data unique? What replaces `TRUNCATE` under parallelism?

22.5 User Isolation — Refined

Theory: User isolation gives each parallel worker its own credentials, session, and test persona to prevent login eviction (second login kills first session), token invalidation, and profile overwrites. Reusing one user across threads guarantees interference. Pattern: key users by `workerIndex` (`user-${workerIndex}`), provision via worker-scoped fixture, clean up after. Combine with isolated browser contexts and per-worker `storageState`. Enterprise Relevance: Single shared `admin` across 8 workers = session thrash + audit-log garbage + false failures. Deloitte pattern: worker-scoped accounts seeded in `beforeAll`, distinct roles per worker where RBAC matrix needed.

```
// worker-scoped fixture: one user per worker, reused by its tests
const userName = `user-${test.info().workerIndex}`;
await createUserInTestDatabase(userName);
// + isolated context + storageState per worker
```

High-Stakes Scenario: SSO allows 1 session/user — 4 workers sharing `qa-admin` log each other out mid-test, random 401s. Triage: 4 users (`qa-admin-w0..w3`), per-worker `storageState` files, no shared login fixture; verify sessions stable across full run. Anti-Patterns: One credential in vault shared by all workers; UI login per test (slow + eviction); `storageState` committed to Git with live tokens. Qs: Why does sharing one user break parallelism even with isolated browsers? How do you provision per-worker users?

22.6 Database Contention — Refined

Theory: Database contention arises when parallel tests fight over rows, locks, sequences, transactions, or connection pools. Concurrent updates cause deadlocks, lock timeouts, dirty reads, and pool exhaustion. Mitigate by partitioning data per worker (schemas `test_worker_N`, tenant-per-run), transactional rollback or per-test schemas, avoiding shared tables, seeding via APIs, and sizing pools for `workers × connections-per-test`. Never share one mutable row across threads. Enterprise Relevance: DB is the hidden parallel ceiling: 16 workers × 5 connections = 80 pooled connections against a 50-connection staging DB = timeout storm. Deloitte capacity math always includes DB pool + app threads, not just CI workers.

```
-- per-worker partition: no cross-worker locks
CREATE SCHEMA test_worker_3; -- seed + teardown per worker
-- pool sizing: workers(8) × conns(5) = 40 ≤ pool(50); else queue
```

High-Stakes Scenario: Parallel suite deadlocks hourly on `orders` sequence + 503 pool-exhausted. Triage: per-worker schemas, UUID PKs (no sequence contention), pool 20→60 with DBA approval, API seeding (fewer open txns), retry-on-deadlock only with backoff + jitter. Anti-Patterns: Shared `test` schema with fixed IDs; sequence PKs under parallel insert; pool sized for serial; `TRUNCATE` mid-run; no lock-timeout (hangs forever). Qs: How do you size DB pools for parallel suites? Partition vs rollback vs shared-seed trade-offs?

22.7 Application Capacity — Refined

Theory: The application under test has finite CPU, memory, threads, queues, and downstream APIs. Over-parallelization turns functional runs into accidental load tests, triggering rate-limiting, 503s, queue backlogs, and timeouts that look like product bugs. SDET must throttle concurrency to app SLOs: use `maxFailures` fail-fast, stagger heavy flows, mock external services, monitor APM. Scale workers gradually and coordinate with performance budgets — functional parallelism respects the app's capacity envelope. Enterprise Relevance: Blasting staging with 32 workers and filing 40 "bugs"

(all 503s) destroys credibility. Deloitte rule: parallel plan reviewed with app team (SLOs, rate limits, test-window capacity); staging $\neq$ prod capacity.

```
export default defineConfig({ workers: process.env.CI ? 2 : 4, maxFailures: 10 });
```

High-Stakes Scenario: Nightly 16-worker run DDoSes staging payment sandbox (rate limit 800/min) — 200 false failures + third-party bill spike. Triage: workers 16→4 for payment specs, WireMock sandbox for third-party, stagger heavy flows, separate perf-window agreement with app team. Anti-Patterns: Max workers against shared staging without asking; no `maxFailures` (burns hours on dead env); real third-party calls at parallel volume; confusing 503-rate-limit with product bug. Qs: How do you distinguish app-capacity failures from real bugs? Who owns the parallelism limit?

22.8 Browser Capacity — Refined

Theory: Each browser instance consumes 200-500MB RAM plus CPU for rendering, video, and DevTools. Local machines saturate at 4-8 parallel browsers; beyond that: OOM kills, slow rendering, flaky waits (rendering starved, not app bugs). Mitigate: headless, `--disable-gpu`, video-off except failures, reusable contexts (not new browser per test). Playwright starts one browser per worker and reuses it. Size `workers` to cores/RAM, not test count; offload to cloud/Grid beyond local ceiling. Enterprise Relevance: Developer laptop (16GB) running 12 headed workers = thrash mistaken for product regression. Deloitte CI standard: `--workers=50%` of cores default, headless, video retain-on-failure; headed debugging local only.

```
npx playwright test --workers=50% # half of CPU cores – safe default
# headed + video + 12 workers on 8GB runner = OOM by design
```

High-Stakes Scenario: CI runner OOM-kills browsers at 70% suite (exit 137), reported as "flaky tests". Triage: `dmesg` OOM evidence, workers 8→4, video off except failure, `--disable-gpu --no-sandbox --disable-dev-shm-usage`, larger runner class for UI shards (cost vs stability math). Anti-Patterns: Workers = test count; headed + video always-on in CI; new browser per test (slow); ignoring exit-137 as flake. Qs: How do you size workers from cores/RAM? What proves OOM vs app failure?

22.9 Grid Capacity — Refined

Theory: Selenium Grid capacity = Hub/Router plus Node slots: `max-sessions`, browser stereotypes (name/version/platform), OS resources. Grid routes WebDriver commands to remote browser instances for cross-browser/platform parallelism. Over-subscribing queues sessions, inflates wait time, and times out (session-queue-full). Plan nodes by `concurrency = nodes × slots`; enable observability/GraphQL (`/graphql` queue length, session count); autoscale with Docker/K8s (HPA/KEDA on queue); shard suites across machines. Enterprise Relevance: Grid queue is the Selenium equivalent of CI queue tax. Deloitte pattern: queue-length dashboard + alert (>5min), autoscale nodes 2→10 on queue depth, TestNG thread-count $\leq$ total slots, session-timeout eviction for hung slots.

```
java -jar selenium-server.jar hub --max-sessions 24
java -jar selenium-server.jar node --max-sessions 8 --hub http://hub:4444
# capacity: 3 nodes × 8 slots = 24 concurrent; threads must be ≤ 24
```

High-Stakes Scenario: 8 threads vs 4 slots — queue grows, sessions time out after 60s, reported as "Grid flaky". Triage: slots $\geq$ threads + buffer, `session-request-timeout` / `retry-interval` tuned,

VNC spot-check for hung browsers holding slots, video-off to free CPU. Anti-Patterns: Threads > slots (permanent queue); no queue monitoring; hung sessions never evicted; single Hub SPOF at 100+ nodes (go distributed). Qs: How do slots, threads, and queue interact? What metric triggers autoscale?

22.10 Worker Count — Refined

Theory: Playwright workers are isolated OS processes, each with its own browser and environment; no inter-worker communication. Playwright reuses workers across files and restarts them after failures for pristine state. Default = half CPU cores locally, typically 2 on CI. Control via `--workers 4` or `workers: process.env.CI ? 2 : undefined`. Files run parallel by default; `fullyParallel: true` or `describe.configure({mode: 'parallel'})` enables intra-file parallelism. Isolate data per worker with `workerIndex / parallelIndex` fixtures and `scope: 'worker'` (unique DB users/accounts). Enterprise Relevance: Worker count is the primary parallelism knob — set from capacity math (cores, RAM, DB pool, app SLO), not guesswork. Deloitte default: CI 2-4 workers, `--workers=50%` local, `--workers=1` to debug order-dependence.

```
// playwright.config.ts
export default defineConfig({ workers: process.env.CI ? 2 : 4, fullyParallel: true
});
```

```
// per-worker account fixture
export const test = base.extend<{}, { dbUser: string }>({
  dbUser: [async ({}, use) => {
    const u = `user-${test.info().workerIndex}`; await use(u);
  }, { scope: 'worker' }]
});
```

High-Stakes Scenario: `--workers=16` on 4-core runner — thrash, 3x slower than 4 workers + flake storm. Triage: `workers ≤ cores` (CPU-bound) or `≤ RAM/500MB` (browser-bound), whichever smaller; measure `S(N)` curve; `--workers=1` to confirm isolation before scaling. Anti-Patterns: Workers = test count; same worker count local and CI (different capacity); worker-scoped state in test-scoped fixtures (leaks across workers). Qs: Workers vs threads vs processes? How do you size workers for a new repo?

22.11 Sharding — Refined

Theory: Sharding splits suites across machines when single-machine workers saturate. `--shard=x/y` : 4 jobs run 1/4 through 4/4 in parallel via GitHub matrix (`shardIndex/shardTotal`), GitLab `parallel` , CircleCI `parallelism` . With `fullyParallel: true` splitting is per-test; otherwise per-file — keep files small and balanced. Emit `blob` reporter per shard, merge centrally (`merge-reports`). Standard Deloitte CI pattern with matrix + merge job. Enterprise Relevance: Sharding is how 1000+ test suites hit <10min PR feedback. Cost scales linearly with shards — balance by timing data, or one giant file defines total time regardless of shard count.

```
npx playwright test --shard=2/4
npx playwright merge-reports --reporter html ./all-blob-reports
```

```
strategy: { matrix: { shard: [1, 2, 3, 4] } }
run: npx playwright test --shard=${{ matrix.shard }}/4 --reporter=blob
```

High-Stakes Scenario: 4 shards, shard 2 takes 25min (giant `checkout.spec.ts`), others 5min — total 25min. Triage: split files by duration (timing-based balancing), `fullyParallel` for per-test split, slowest-shard dashboard; never add a 5th shard before splitting the giant file. Anti-Patterns: File-count balancing (duration varies 10x); no merge job (4 unreadable reports); sharding order-dependent tests (different shards, different failures); giant single file. Qs: Workers vs shards — when each? What defines sharded total time?

22.12 ThreadLocal — Refined

Theory: Selenium WebDriver and Playwright-Java objects are not thread-safe; sharing one static driver across parallel TestNG/JUnit threads causes session conflicts, `SessionNotFoundException`, and flaky overwrites (test A drives test B's browser). `ThreadLocal<WebDriver>` gives each thread an isolated copy via `get()/set()/remove()`. Initialize in `@BeforeMethod`, quit and `remove()` in `@AfterMethod` (pooled threads reuse → leak without remove). Page Objects receive `ThreadLocal.get()` driver, never store shared static state. Playwright Node avoids this via separate worker processes (no shared JVM). Enterprise Relevance: The canonical Selenium-parallel pattern — every Deloitte TestNG framework must show it. Missing `remove()` leaks drivers until Grid slots exhaust mid-suite.

```
private static final ThreadLocal<WebDriver> DRIVER = new ThreadLocal<>();
@BeforeMethod public void setUp() {
    DRIVER.set(DriverFactory.create(Config.browser())); }
public static WebDriver get() {
    if (DRIVER.get() == null) throw new IllegalStateException("No driver on " +
        Thread.currentThread().getName());
    return DRIVER.get();
}
@AfterMethod(alwaysRun = true) public void tearDown() { try { if (DRIVER.get() !=
    null) DRIVER.get().quit(); } finally { DRIVER.remove(); } }
```

High-Stakes Scenario: Parallel suite, tests navigate each other's pages randomly. Triage: static driver found → `ThreadLocal` migration; `ThreadGuard.protect(driver)` fails fast on cross-thread access; verify with thread-name logging per action. Anti-Patterns: Static shared driver; `set()` without `remove()`; storing `ThreadLocal` inside Page Objects; passing driver through 5 constructors instead of manager. Qs: Why is `remove()` mandatory with thread pools? How does Node Playwright avoid this class entirely?

22.13 Parallel Test Design — Refined

Theory: Effective parallelism requires atomic, autonomous, order-independent tests with just-in-time isolated data. Avoid shared accounts, global counters, fixed ports, common files, test-to-test dependencies. Prefer unique UUIDs, per-worker databases (`testdb_worker_${parallelIndex}`), dynamic ports, `storageState` auth over repeated UI logins. Suite time equals slowest shard — keep slowest test short. Isolate via fresh `BrowserContext` per test with explicit setup/teardown, never serial chains. Enterprise Relevance: Design-for-parallelism is cheaper than fixing serial-assumed suites later. Deloitte checklist for every new test: independent? unique data? no fixed ports/files? no order dependence? passes with `--workers=4` and `--repeat-each=3`?

```
test.describe.configure({ mode: 'parallel' });
const user = `user-${randomUUID()}`; // unique, not shared
```

```
const db = `testdb_worker_${test.info().parallelIndex}`; // per-worker namespace
// dynamic port: const port = 3000 + test.info().workerIndex;
```

High-Stakes Scenario: New tests use fixed port 3000 + shared `seed.sql` — parallel run: port collisions + data overwrites, 40% flake on new tests only. Triage: dynamic ports, per-worker DBs, API seeding per test, `fullyParallel` CI job that must pass before merge. Anti-Patterns: Fixed ports/paths; shared seed file mutated per test; `test.describe.serial` as default; UI login per test (slow) instead of `storageState`. Qs: What makes a test parallel-safe by construction? How do you review new tests for parallel hazards?

22.14 Measuring Speedup — Refined

Theory: Speedup $S(N)=T(1)/T(N)$; efficiency $E=S/N$. Amdahl's Law $S=1/((1-p)+p/N)$ bounds speedup by serial fraction p — even 5% serial limits 16 workers to ~9x. Measure wall-clock across 1..N workers, average multiple runs, plot speedup vs ideal linear scaling. Derive parallel fraction from observed data; track per-file durations, shard-balance variance, CI overhead (setup, browser install, report merging) to justify worker counts with data, not hope. Enterprise Relevance: "Add more workers" without measurement wastes money and hides serial bottlenecks (usually setup + slowest file + report merge). Deloitte capacity reviews require the S-curve chart before approving bigger runners.

```
T1=600s; T4=180s; S=600/180 # 3.3x on 4 workers, E=0.83 — healthy
T16=70s; S=600/70 # 8.6x on 16 workers, E=0.54 — diminishing; find serial
fraction
# Amdahl: p=0.05 → S(16)=1/(0.05+0.95/16) ≈ 9.1x ceiling
```

High-Stakes Scenario: 16 workers barely faster than 8 (8.6x vs 7.9x) yet cost 2x. Triage: profile serial fraction (global setup 40s + slowest file 50s + merge 20s = 110s serial floor), split slowest file, cache setup, parallelize merge. Efficiency <0.5 = fix serial before adding workers. Anti-Patterns: Quoting worker count as speedup; single measurement (noise); ignoring CI overhead (install + merge); linear extrapolation to 100 workers. Qs: Compute S and E from $T1/TN$? What does $E<0.5$ tell you? Where is your suite's serial floor?

22.15 Diminishing Returns — Refined

Theory: Beyond an optimum, added workers increase contention for CPU, memory, IO, database connections, and network — completion-time curves flatten then stall (Microsoft documents minimum then stall). Local workers are capped by cores; remote browsers still stress client machines (bandwidth, file handles). Symptoms: rising flakiness, CPU thrashing, port collisions, longer wall-clock with more workers. Mitigate: size workers to cores, scale runners (not just workers), shard across machines, cache binaries, mock third parties, re-profile rather than blindly adding parallelism — compensate fewer workers with more shards. Enterprise Relevance: The "just add workers" reflex burns budget while flake rises. Deloitte rule: worker increase requires S-curve evidence; otherwise fix contention (DB pool, app capacity, shard balance) first.

```
// compensate fewer workers with more shards (different bottlenecks)
export default defineConfig({ workers: 2 }); // + 8 shards across machines
// vs workers: 16 on 4 cores — thrash, not speed
```

High-Stakes Scenario: Workers 4→16, suite 12min→14min + flake 2→9% (DB pool 20 connections, app rate limit). Triage: workers back to 4, shards 1→4 across runners, DB pool 20→60, third-party mocked,

re-measure S-curve. Speed from sharding (separate DBs/hosts), not workers (shared everything). Anti-Patterns: More workers as first fix; ignoring contention signals (CPU steal, pool waits, 429s); comparing worker counts across different hardware. Qs: Workers vs shards vs runners — which bottleneck does each relieve? What signals say "fewer workers"?

22.16 Failure Rate Under Concurrency — Refined

Theory: Concurrency exposes hidden shared mutable state, causing order-dependent and atomicity failures: duplicate records, dirty reads, deadlocks, data races, async-wait timeouts. Studies show concurrency, async-wait, and order-dependency dominate flakiness; Google reports ~16% tests flaky with 84% of pass-to-fail transitions caused by flakes. Measure via repeated reruns (`--repeat-each`) or stressors (SHAKER). Fix with per-worker isolation, transactions with rollback, locks for unavoidable sharing, deterministic waits, quarantine pipelines, retry analytics. Enterprise Relevance: Concurrency failure rate is the quality metric for parallel suites — track it separately from single-threaded pass rate. A suite 99% serial / 90% parallel has an isolation problem, not a product problem.

```
beforeEach(async () => { tx = await db.beginTransaction(); });
afterEach(async () => { await tx.rollback(); }); // no cross-test residue
// measure: npx playwright test --repeat-each=10 --workers=4 → flake rate per test
```

High-Stakes Scenario: Release gate 92% (8% "flake"), team ships anyway — 3 of the 8% are real order-dependent product bugs (discount stacking). Triage: quarantine + classify each flake (isolation vs timing vs env vs real), fix isolation class first (kills most), re-run measurement; gate on concurrency failure rate, not just serial pass. Anti-Patterns: Averaging flake across suite (hides worst tests); retrying without classifying; calling concurrency failures "infrastructure" without evidence. Qs: How do you separate isolation failures from timing failures? What flake rate blocks release?

22.17 10,000-Test Suite Case Study — Refined

Theory: Google scale: 4.2M tests execute 150M times daily via Bazel sharding, regression-test selection (RTS — only affected tests), and milestone batching; only 1.23% ever find breakage. Doctolib: 14K browser E2E with Kubernetes-native Cirrus, up to 832 parallel jobs with sidecar Postgres/Elasticsearch/Redis per job. QALadder: 10K tests in 8:40 using 42 duration-balanced containers — after fixing one shared staging database with per-worker namespacing (the single change that unlocked scale). Enterprise Relevance: The lesson sequence is fixed: (1) isolate data first, (2) bin-pack by historical duration, (3) then scale shards. Teams that scale before isolating multiply interference, not throughput. Deloitte 10k design review always starts with data architecture, not worker counts.

```
npx playwright test --shard=$CI_NODE_INDEX/$CI_NODE_TOTAL # + RTS: only changed-
area tests on PR
# QALadder math: 10,000 tests / 42 containers ≈ 238 each × ~2.2min ≈ 8:40 wall
```

High-Stakes Scenario: "We need 10k tests in 10min" with shared staging + 200-line `beforeAll` login + 1 giant file. Triage: per-worker namespaces (unblock), API seeding (kill login tax), file split by duration, RTS on PR (10% of suite), full nightly. Realistic path: 10min PR slice + 30min nightly full — not 10k in 10min on day one. Anti-Patterns: Scaling broken isolation ("more shards will fix flakes"); duration-blind sharding; shared staging at 10k scale; quoting Google numbers without Google infra. Qs: Why does isolation precede sharding? What are the three scaling stages in order?

22.18 Scaling Interview Questions — Refined

Deloitte SDET interviews probe ownership beyond syntax: workers vs shards (processes on one machine vs suite split across machines; total parallelism = shards × workers), blob report merging, GitHub matrix design. Expect capacity math (500 tests across 4 shards × 4 workers to hit SLA minus 30-60s overhead per shard), duration-based bin-packing, isolation-first strategy, storageState reuse, API-over-UI shifts, flake quarantine. Prepare concrete numbers: runtime before/after, flake rate, infra cost + trade-offs (queues, autoscaling spot fleets, test selection).

```
strategy: { fail-fast: false, matrix: { shard: [1/4, 2/4, 3/4, 4/4] } }  
run: npx playwright test --shard=${{ matrix.shard }}  
# interview math: 500 tests × 30s avg = 250min serial; 4×4 parallel ≈ 250/16 +  
overhead ≈ 18min
```

Answer template: "500 tests, 30s avg → 250min serial. 4 shards × 4 workers = 16x theoretical, ~13x actual after overhead → ~19min. Bottlenecks: DB pool (sized 60), slowest file (split), flake 1.5% (quarantined 3). Cost: 4 runners × 20min." Anti-Patterns: "Add more workers" without math; confusing workers/shards; no cost awareness; ignoring overhead/flake in estimates. Qs: 500 tests, 30s each, 10min SLA — design it? Workers vs shards vs runners?